

A Hybrid Gradient Boosting Framework for Interpretable Credit Card Fraud Detection

Bandikolla Rahul

Dept of CSE(AI&ML)

Vardhaman College of Engineering

Hyderabad, India

rahulbandikolla@gmail.com

Bumandla Rajashekar

Dept of CSE(AI&ML)

Vardhaman College of Engineering

Hyderabad, India

bumandlarajashekar@gmail.com

Uppala Vinay

Dept of CSE(AI&ML)

Vardhaman College of Engineering

Hyderabad, India

uppalavinay5678@gmail.com

Koti Tejasvi

Dept of CSE(AI&ML)

Vardhaman College of Engineering

Hyderabad, India

kotitejasvi@gmail.com

M. A. Jabbar

Dept of CSE(AI&ML)

Vardhaman College of Engineering

Hyderabad, India

jabbar.meerja@gmail.com

Abstract—Less than a fifth of a percent of credit card transactions in a typical dataset are fraudulent, and that imbalance alone makes reliable detection hard – on top of it, fraud patterns keep changing as the people committing it adapt to whatever a bank deploys against them. This paper walks through the implementation and evaluation of a complete, end-to-end fraud detection system built around an ensemble of three gradient-boosted tree models: XGBoost, LightGBM, and CatBoost, tuned with Tree-structured Parzen Estimator optimization. The system runs behind a Flask REST API and a React dashboard, and every prediction the dashboard shows comes with a SHAP-based explanation of which features drove it. We describe the full pipeline, report an ensemble PR-AUC of 0.8481 on 5-fold cross-validation and 0.8192 on a stratified 15% held-out test set never seen during training or hyperparameter selection, and show that the live API reproduces offline scores essentially exactly. We also cover a design path we tried and abandoned: a GraphSAGE-based graph neural network that, once tested properly, turned out to underperform the simpler ensemble – a result consistent with known heterophily and camouflage effects in fraud transaction graphs. The gap between what the offline experiments predicted and what the deployed API actually returns turned out to be negligible, which is not something every fraud-detection paper checks.

Index Terms—Fraud Detection, Gradient Boosting, Ensembles, Hyperparameter Optimization, Graph Neural Networks, SHAP

I. INTRODUCTION

Digital payments keep growing, and the fraud riding along with them keeps getting harder to catch. Any system meant to catch it has to score huge volumes of high-dimensional transaction data fast, and it has to do that knowing that only a sliver of a percent of what it sees is actually fraud [1], [2]. Rule-based engines used to be the standard approach; they have mostly been replaced by learned models, and among those, tree-based boosting methods have held up especially well [3], [4].

This paper covers the implementation of an ensemble-based fraud detection system, start to finish – data engineering, hyperparameter search, ensembling, the API that serves it, and

the dashboard sitting on top. The core of the system is an average-probability ensemble of XGBoost [5], LightGBM [6], and CatBoost [7]. Every prediction the dashboard shows a user is accompanied by a SHAP [8] breakdown of which features pushed the score toward fraud or away from it.

We also document something most papers in this area leave out: a design alternative we built, tested, and rejected. Before settling on the gradient-boosting ensemble, we built a structural Graph Neural Network approach using GraphSAGE [9]. It underperformed once we ran the ablation properly, and Section IV covers why, tying the result to the graph heterophily and camouflaged-fraudster literature [10], [11].

The rest of the paper is organized as follows. Section II covers related work. Section III walks through the system architecture. Section IV is the graph ablation. Section V is the evaluation, including the deployment validation. Sections VI and VII are discussion, limitations, and conclusion.

II. RELATED WORK

Credit card fraud detection via data mining has been surveyed extensively elsewhere [2], [3], so we won't repeat that ground here. The dataset we use, the European credit card fraud set [1], shows up throughout this literature specifically because it's a good testbed for imbalance-handling techniques – SMOTE [12] being the most common one applied to it.

XGBoost [5], LightGBM [6], and CatBoost [7] are, at this point, the default choice for tabular problems in both research and production, mostly because they pick up non-linear feature interactions on their own instead of requiring someone to hand-engineer feature crosses. Tuning them by hand is tedious, though, which is where automated search frameworks like Optuna [13] come in – it uses Tree-structured Parzen Estimators [14] under the hood, and it's become close to a standard tool for this kind of work.

Graph Neural Networks are the other thread worth covering. Graph Convolutional Networks [15] and GraphSAGE [9] in

particular have found use in anomaly detection. There’s a catch, though: these architectures generally assume that connected nodes tend to share a label – homophily. Fraud doesn’t cooperate with that assumption. Fraudulent accounts often connect themselves to legitimate ones on purpose, precisely to look less suspicious, and the literature has a name for this: heterophily, or camouflage [10], [11], [16]. That mismatch is exactly why we ran the comparison in Section IV.

III. SYSTEM ARCHITECTURE

The system is one pipeline end to end, from raw transaction records to a live dashboard someone can actually click through.

A. Data Pipeline and Feature Engineering

Training data comes from the Kaggle European credit card fraud dataset – 28 PCA components ($V1$ through $V28$), plus *Time* and *Amount*, plus the (heavily skewed) *Class* label. Before any modeling happens, the pipeline deduplicates the raw data and derives two engineered features:

- 1) **Temporal cyclicity:** *Time* gets mapped onto a 24-hour cycle and encoded trigonometrically as *hour_sin* and *hour_cos*, so the model sees “close to midnight” as close regardless of which side of the day boundary a transaction falls on.
- 2) **Magnitude scaling:** *Amount* goes through a $\log(1 + x)$ transform (*log1p_Amount*) to pull in its long tail.

All 33 resulting features get normalized with a scaler [17] fit only on the training folds – fitting it on anything else would leak test-set statistics into training.

B. Cross-Validation Protocol and Optuna HPO

Evaluation runs on 5-fold stratified cross-validation. For each model, a single Optuna [13] study searches hyperparameters against the mean validation PR-AUC across those same five folds – 76 trials for XGBoost, 55 for LightGBM, and 33 for CatBoost.

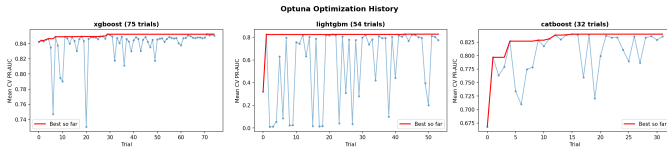


Fig. 1. Optuna search trajectories for all three models. Best-so-far PR-AUC is overlaid on each trial’s raw score.

All three converge quickly, as Fig. 1 shows, though LightGBM’s climb out of a weak starting point is the steepest of the three – its default configuration was genuinely bad, and Section VI gets into why that matters.

C. Ensembling and Model Export

The deployed model is a simple, unweighted average of the three tuned models’ output probabilities: $P = (P_{xgb} + P_{lgb} + P_{cat})/3.0$. We picked a decision threshold of 0.7725 by maximizing F1 on out-of-fold predictions. Everything needed

to reproduce a prediction outside the notebook – the fitted scaler, the exact 33-feature column order, and all three tuned models – gets serialized together for deployment.

D. Flask API and React Dashboard

A Python Flask [18] REST API sits behind the dashboard, with three endpoints doing the actual work:

- `/api/predict` – takes a single transaction, engineers *hour_sin*, *hour_cos*, and *log1p_Amount* on the fly, forces the 33 columns into the exact order training used, scales them, and returns the ensemble’s probability.
- `/api/batch-predict` – the same thing, but for a whole CSV at once.
- `/api/stats` and `/api/model-info` – pull aggregated numbers and metadata straight from the CV summary files, nothing computed on the fly.

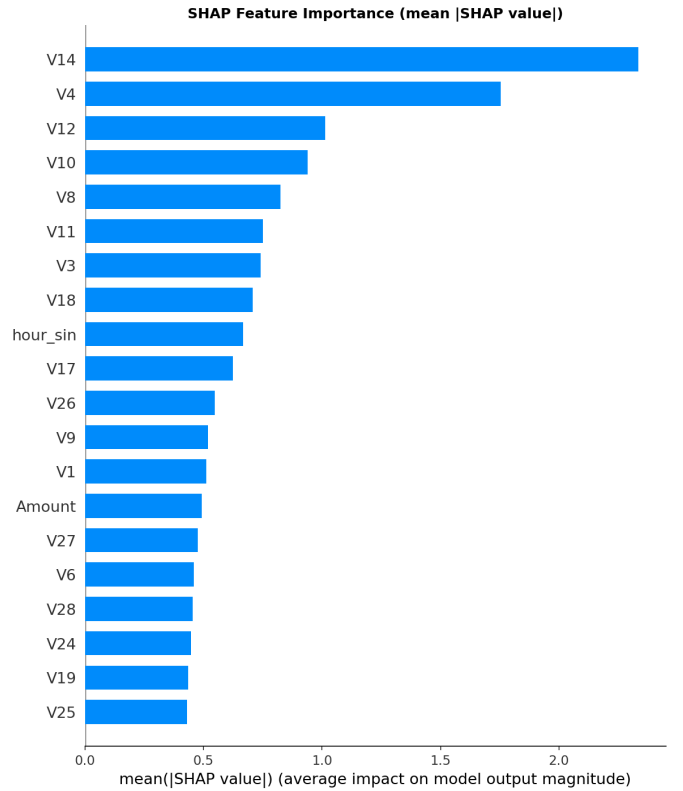


Fig. 2. Global SHAP feature importance for the tuned ensemble (proxied via XGBoost). $V14$, $V4$, and $V12$ lead by a wide margin.

On the frontend side, it’s a single-page React [19] app. A SHAP TreeExplainer [8] runs inside the API and returns per-transaction feature contributions alongside every prediction, which the dashboard renders as a “Why Flagged” bar chart – red bars pushing a transaction toward fraud, green ones pulling it back toward genuine.

IV. DESIGN ALTERNATIVES CONSIDERED: THE GRAPH ABLATION

Before settling on the ensemble described above, we built and seriously evaluated a structural graph approach – inter-

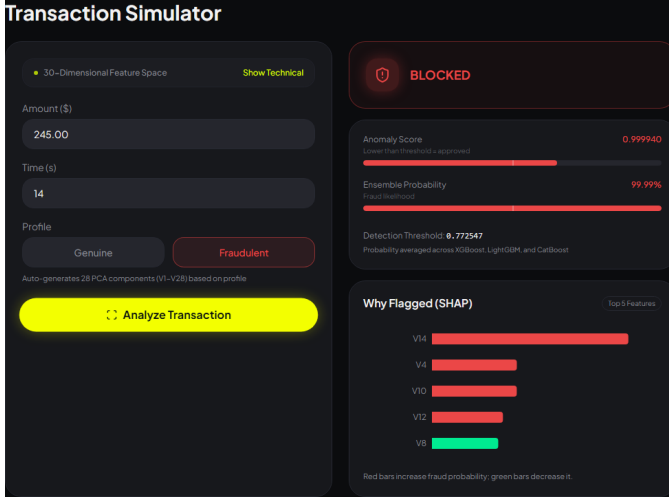


Fig. 3. The simulator scoring a fraudulent transaction live. Anomaly score 0.9999 against a threshold of 0.7725 – the result is a hard BLOCKED classification, not a borderline one.

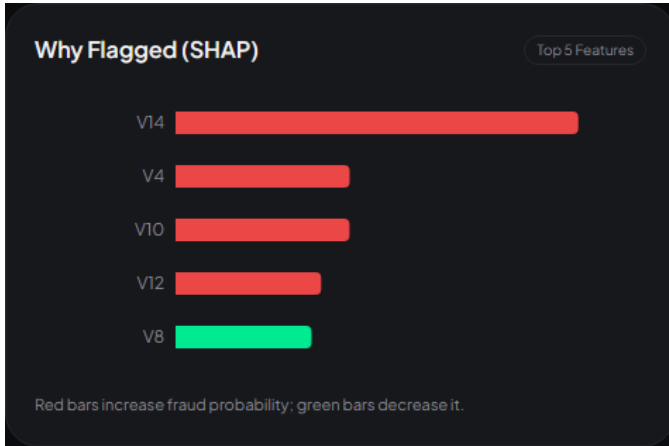


Fig. 4. The per-transaction explanation for that same fraud case. V_{14} , V_4 , V_{10} , and V_{12} line up with the global ranking in Fig. 2; V_8 is the one feature pulling in the opposite direction.

nally, we just called it “FraudGuard.” The idea was to treat transactions as nodes in a k -Nearest Neighbors graph, run GraphSAGE over that graph to get structure-aware embeddings, and feed those embeddings into an Autoencoder trained to flag anomalies by reconstruction error.

The question we actually needed answered was narrower than “does this work”: did the GraphSAGE aggregation step add anything over a plain Autoencoder running on the same tabular features with no graph at all? We compared the full FraudGuard pipeline against that ablated PlainAE baseline across 5 cross-validation splits to find out.

The answer was no – worse than no, actually. FraudGuard’s mean PR-AUC came in at 0.3285 (± 0.0411) against PlainAE’s 0.3879 (± 0.0495), a delta of -0.0594 with Cohen’s $d = -1.345$. ROC-AUC told the same story: -0.0195 , $d = -1.146$. Figs. 5 and 6 lay this out split by split, and it’s consistent, not a fluke on one partition.

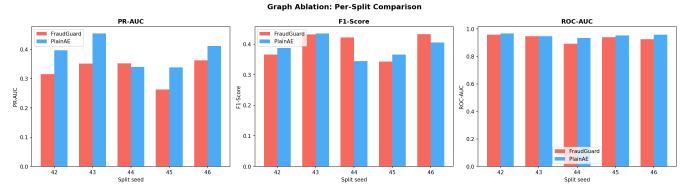


Fig. 5. FraudGuard against the ablated Autoencoder, split by split. PR-AUC and ROC-AUC both come out lower with the graph component included.

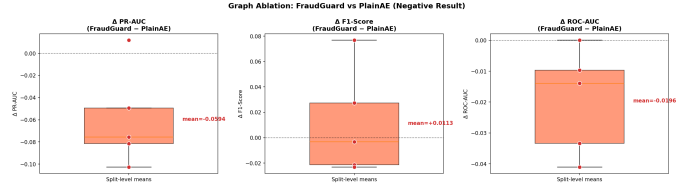


Fig. 6. Per-split deltas, PlainAE minus FraudGuard. Everything below zero means the graph made things worse on that split.

This isn’t a surprising result once you look at what’s known about GNNs in adversarial settings. GraphSAGE and similar message-passing methods assume nearby nodes look alike; fraud actively works against that by camouflaging itself among legitimate transactions, and the aggregation step ends up smoothing away exactly the signal it was supposed to sharpen [11], [16]. We dropped the graph component and moved forward with the plain tabular ensemble.

V. EVALUATION

A. Cross-Validation Results

Fig. 7 lays out how each tuned model performed against the final ensemble, averaged across the same 5 CV folds used everywhere else in this paper.

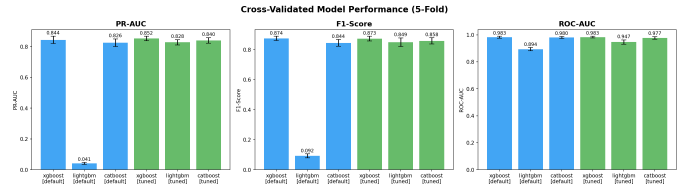


Fig. 7. PR-AUC, F1, and ROC-AUC for every model at both default and tuned settings, plus the ensemble. LightGBM’s default-to-tuned jump is the largest by a wide margin.

The ensemble lands at 0.8481 PR-AUC, 0.8600 F1, 0.9799 ROC-AUC – 384 true positives, 36 false positives, 89 false negatives. Averaging the three models’ probabilities together mostly buys variance reduction: no single model’s idiosyncrasies dominate the final score.

B. Explainability

Fig. 8 is the global view of what the SHAP values look like across the whole feature set. V_{14} , V_4 , and V_{12} are doing most of the work, unsurprisingly, but $hour_sin$ – the engineered feature – cracks the top 10, which is a decent sign

that the temporal-cyclicity step in Section III-A was worth adding rather than just extra complexity.

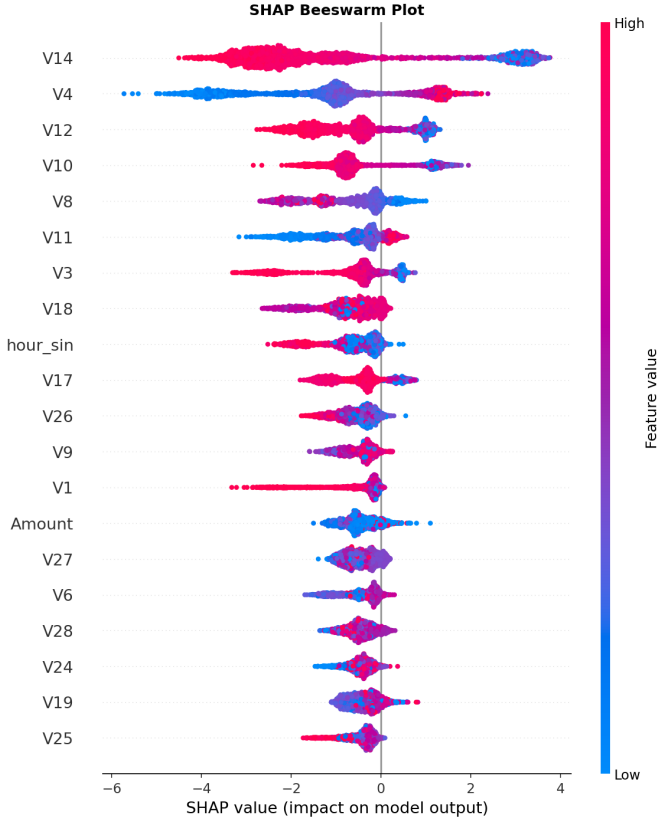


Fig. 8. SHAP beeswarm across the full feature set. High V4 pushes toward fraud; high V14 pushes the other way.

C. Deployment Validation

One of the more common ways an ML system quietly breaks in production is a mismatch between what the offline pipeline does and what the deployed API actually does – a scaler fit differently, columns in a different order, a feature computed at the wrong point in the pipeline. We checked for exactly that: 10 transactions (5 fraud, 5 genuine), scored two ways – once directly in Python against the serialized offline models, and once by sending the identical raw features to the live `/api/predict` endpoint over HTTP.

All 10 matched, with deltas in the 10^{-7} range – floating-point noise, nothing more. That tells us the feature engineering and column ordering are happening in the same sequence in both places, and that the live model is seeing the same 33-column array the offline pipeline saw. It’s worth being upfront about the limit here, though: none of these 10 rows sit near the 0.7725 threshold, so this checks pipeline fidelity, not necessarily behavior at the decision boundary specifically.

D. Held-Out Test Set Validation

The cross-validation metrics reported above were computed on the same folds used to select hyperparameters – a source of mild optimistic bias acknowledged in Section VI. To get a

TABLE I
DEPLOYMENT SANITY CHECK: OFFLINE VS. API SCORES

Id	True Class	Offline Score	Online API Score	Delta (Δ)
0	Fraudulent	0.9999030	0.9999030	7e-08
1	Fraudulent	0.9745024	0.9745024	4e-07
2	Fraudulent	0.9998607	0.9998607	2e-07
3	Fraudulent	0.9998690	0.9998690	7e-08
4	Fraudulent	0.9999244	0.9999244	4e-07
5	Genuine	0.0000168	0.0000168	1e-07
6	Genuine	0.0000134	0.0000134	4e-07
7	Genuine	0.0000540	0.0000540	6e-08
8	Genuine	0.0000251	0.0000251	1e-07
9	Genuine	0.0000311	0.0000311	1e-07

number that isn’t touched by that bias at all, we carved out a stratified 15% holdout (42,559 transactions, 71 fraudulent) before anything else in the pipeline ran. The three models were retrained once on the remaining 85% using the same tuned hyperparameters already selected by Optuna, with a fresh scaler fit only on the development set, and scored on the holdout with the same ensemble averaging and 0.7725 threshold.

TABLE II
HELD-OUT TEST SET RESULTS (ENSEMBLE, $n = 42,559$)

Metric	Value
PR-AUC	0.8192
F1	0.8527
ROC-AUC	0.9669
Precision	0.9483
Recall	0.7746
TP / FP / TN / FN	55 / 3 / 42,485 / 16

The holdout PR-AUC of 0.8192 (Table II) sits about 0.03 below the CV figure of 0.8481, which is roughly what you’d expect given the fold-overlap bias discussed in Section VI. The drop is real but modest – the system isn’t dramatically overfitting to the CV folds, it’s just getting a small free ride from having seen the same partitions during both tuning and evaluation. The holdout F1 (0.8527) and ROC-AUC (0.9669) tell the same story: slightly lower than CV, but in the same ballpark.

VI. DISCUSSION AND LIMITATIONS

The clearest lesson from this project is how much hyperparameter tuning can matter, and how unevenly it matters across models. LightGBM’s default configuration was close to useless on this data – PR-AUC of 0.0407 – and TPE tuning brought it up to 0.8281, the single largest jump in the whole comparison (Fig. 7).

The ensemble itself is a mixed story, and we’d rather say that plainly than oversell it. A paired Wilcoxon signed-rank test across the 5 CV folds – the same method used for the graph ablation in Section IV – gives a mean PR-AUC delta of -0.0035 (ensemble minus XGBoost), Cohen’s $d = -0.638$,

and $p = 0.3125$. XGBoost wins on 4 of the 5 folds, and the graduated-verdict rule flags this as “tends to degrade.” In plain terms: the ensemble doesn’t improve on the best single model by PR-AUC, and if anything it sits slightly below it. What the ensemble buys instead of raw accuracy is variance reduction – it’s less likely to fail badly on the specific edge cases any one of the three models handles poorly, even if its average-case PR-AUC is a fraction of a point worse.

One limitation worth flagging directly: hyperparameters were chosen by maximizing performance on the same five folds we then used to report the CV metrics in Section V-A. That’s a real source of mild optimistic bias, and the held-out results in Section V-D (PR-AUC 0.8192 vs. the CV’s 0.8481) confirm it – the gap is modest but real. The held-out evaluation addresses this limitation for the purposes of reporting an unbiased generalization estimate, though a fully nested cross-validation protocol would be the more rigorous long-term fix. A second limitation: for latency reasons, the live SHAP explanations come from XGBoost alone, used as a stand-in for the full ensemble. Gradient-boosted models tend to agree on decision boundaries in broad strokes, but there will be edge cases where LightGBM or CatBoost is actually driving the score and XGBoost’s explanation doesn’t quite match what’s really happening.

VII. CONCLUSION

We built and deployed a fraud detection system around an XGBoost/LightGBM/CatBoost ensemble, and along the way we tested – and rejected – a graph-based alternative once the numbers showed it degrading performance under exactly the heterophily and camouflage conditions the literature predicts for this kind of data. The deployment validation confirms the live API isn’t just close to the offline numbers, it reproduces them to floating-point precision. The system achieves a CV PR-AUC of 0.8481 and a held-out PR-AUC of 0.8192 on data never seen during training or hyperparameter selection, and gives a SHAP-based explanation for every transaction it scores.

REFERENCES

- [1] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, “Calibrating probability with undersampling for unbalanced classification,” in *2015 IEEE symposium series on computational intelligence*. IEEE, 2015, pp. 159–166.
- [2] A. Abdallah, M. A. Maarof, and A. Zainal, “Fraud detection systems: A survey,” *Journal of Network and Computer Applications*, vol. 68, pp. 90–113, 2016.
- [3] C. Phua, V. Lee, K. Smith, and R. Gayler, “A comprehensive survey of data mining-based fraud detection research,” *arXiv preprint arXiv:1009.6119*, 2010.
- [4] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, “Data mining for credit card fraud: A comparative study,” *Decision Support Systems*, vol. 50, no. 3, pp. 602–613, 2011.
- [5] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.
- [6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and L. Tie-Yan, “Lightgbm: A highly efficient gradient boosting decision tree,” *Advances in neural information processing systems*, vol. 30, 2017.
- [7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “Catboost: unbiased boosting with categorical features,” in *Advances in neural information processing systems*, vol. 31, 2018.

- [8] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in neural information processing systems*, vol. 30, 2017.
- [9] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in neural information processing systems*, vol. 30, 2017.
- [10] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, “Beyond homophily in graph neural networks: Current limitations and effective designs,” in *Advances in neural information processing systems*, vol. 33, 2020, pp. 7793–7804.
- [11] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, “Enhancing graph neural network-based fraud detectors against camouflaged fraudsters,” in *Proceedings of the 29th ACM international conference on information & knowledge management*, 2020, pp. 315–324.
- [12] R. Blagus and L. Lusa, “Smote for high-dimensional class-imbalanced data,” *BMC bioinformatics*, vol. 14, no. 1, pp. 1–16, 2013.
- [13] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2623–2631.
- [14] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyperparameter optimization,” in *Advances in neural information processing systems*, vol. 24, 2011.
- [15] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [16] Y. Liu, X. Ao, Z. Dong, J. Chao, J. He, K. Bian, and Q. Feng, “Alleviating the inconsistency problem of applying graph neural network to fraud detection,” in *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*, 2021, pp. 1569–1578.
- [17] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, “Scikit-learn: Machine learning in python,” *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.
- [18] M. Grinberg, *Flask web development: developing web applications with python*. O’Reilly Media, Inc., 2018.
- [19] Facebook, “React: A javascript library for building user interfaces,” <https://reactjs.org/>, 2013.